



DATA ANALYTICS

RNCP Level 6 bloc 1

Defence Report

MVP 1 & MVP 2 — Tiny House Project

From Territorial Analysis to a Social Mobility Strategy

Veronique FANCHONNA

Data Analyst — RNCP Level 6 bloc 1

February 2026

"From executing analyst to data infrastructure auditor"

Table of Contents

MVP 1 — Tiny House Project: Mapping the Sweet Spots

- I. Introduction and Strategic Framework
- II. Technical Architecture and Methodology
- III. Technical Challenges and Analyst Resilience
- IV. Results Analysis and Data-Driven Insights
- V. Conclusion and Transition to MVP 2

MVP 2 — From Mapping to Prediction

- VI. Introduction and Strategic Vision of MVP 2
- VII. Architecture and Methodology
- VIII. Lessons Learned: Challenges and Resilience
- IX. Results Analysis and Data-Driven Insights
- X. Conclusion and MVP 3 Roadmap
- XI. REST API Implementation & Data Accessibility

MVP 1

Mapping the "Sweet Spots" Real Estate Data Analysis across France

I. Introduction and Strategic Framework (The "Why")

1.1. Project Origins: Accessibility as a Driver of Emancipation

This project was not born from mere analytical curiosity, but from a deep conviction: access to decent and affordable housing is the primary lever of social emancipation. In a context where low-income households spend a growing share of their income on rent — often beyond the recommended 30% effort rate — modular housing (Tiny Houses, converted containers) presents itself as a credible alternative to the traditional rental model.

My approach follows a dual dynamic: on one hand, financial emancipation through a reduction in housing costs within the household budget; on the other hand, the recovery of geographical and professional mobility. The issue is not simply to "pay less for housing", but to free up "remaining income" to allow investment in education, training, or entrepreneurship.

1.2. Initial Objective of MVP 1: Defining the Geographical "Sweet Spot"

MVP 1 pursues a clear operational objective: to identify French municipalities where the "land price / access to services" equation is affordable. This search for a "Sweet Spot" is based on the hypothesis that there are areas in France where land acquisition would cost $\leq 50\text{€}/\text{m}^2$ and would allow a viable modular housing project, without sacrificing access to essential services (education, healthcare, transport and therefore employment).

Three target profiles guide my segmentation:

- Young urban workers: Looking for proximity to employment + mobility, targeting micro-plots ($< 50\text{m}^2$)
- Single-parent families or couples: Prioritising schooling + healthcare, targeting standard plots ($50\text{--}5,000\text{m}^2$)
- Remote workers: Accepting distance in exchange for quality of life, open to large properties ($> 5,000\text{m}^2$)

1.3. Business Stakes: From Imposed Housing to Chosen Housing

This project directly challenges the paradigm of "imposed housing" driven by market pressure. By mapping affordable land opportunities combined with sufficient infrastructure for working people, I propose a decision-support tool for households wishing to move from the status of precarious renter to that of low-cost homeowner. The ultimate goal is the reclaiming of control over one's life trajectory — in terms of financial, family, and professional capacity.

II. Technical Architecture and Deployed Methodology

2.1. Data Ecosystem (Data Sourcing)

The robustness of this project relies on the triangulation of three complementary public data sources, whose integration forms the technical core of MVP 1.

2.1.1. DVF Database (Property Value Requests): Establishing Real Prices

The DVF is the factual foundation of my market analysis, with 18 million+ records of real estate transactions across the country. Unlike algorithmic estimates from property portals, the DVF exposes the prices actually paid, ensuring that my analyses are grounded in transactional reality.

Key extracted variables:

- valeur_fonciere: transaction amount
- transaction type: sales only (avoiding gifts, legacies, exchanges,...)
- surface_terrain: property area (built-up only)
- nature_culture: land type (residential plot / habitable)
- code_commune (INSEE): relational key with other databases (zip code)

2.1.2. BPE Database (Permanent Equipment Survey): Measuring Territorial Attractiveness

The BPE lists all facilities and services present on French territory, with a granularity down to the neighbourhood level (IRIS). For this project, I filtered the categories essential to daily viability:

- Primary education (nursery and primary schools)
- Healthcare (general practitioners, pharmacies)
- Public transport (train stations, bus stops)
- Essential commerce (supermarkets, bakeries, retails)

Deliberate exclusions: Taxis (not representative of accessible mobility) and secondary school levels (middle school, high school).

2.1.3. INSEE Reference: The Interoperability Pivot

The 5-digit INSEE code plays the role of a universal key enabling the join between DVF and BPE datasets.

2.2. Data Engineering Pipeline (df_ to _db Convention)

I adopted a strict naming convention to differentiate processing environments:

- Prefix `df_`: Pandas objects in memory (Python DataFrames)
- Suffix `_db`: Tables persisted in MySQL

2.2.1. Python Processing (Pandas): Cleaning, Typing and Aggregation

Phase 1 — Cleaning:

- Removal of null values on `valeur_fonciere` and `surface_terrain` (3–5% of the data)
- Calculation of price per m²: `df['prix_m2'] = df['valeur_fonciere'] / df['surface_terrain']`
- Area categorisation: Micro (< 50m²), Standard (50–5,000m²), Large (> 5,000m²)

Phase 2 — Statistical Aggregation:

- Calculation of median price per municipality and surface category (robust to outliers)
- Calculation of IQR (Interquartile Range) to assess price dispersion AND remove outliers (notably Paris with extreme prices)
- Filter on `nb_transactions` ≥ 5 to guarantee statistical reliability

2.2.2. SQL Persistence: Structuring Tables in MySQL

The cleaned DataFrames were exported to MySQL via SQLAlchemy with the following schema:

- `dvf_final_db`: DVF data (FK: `insee_code`)
- `bpe_commune_db`: Aggregated BPE data by municipality (FK: `insee_code`)
- `communes_db`: INSEE reference table extract with API (FK: `insee_code`)
- `df_dept`: List of departments (number + name) in webscraping

2.3. Join Strategy: Unification via INSEE Code

The standardisation of the INSEE code as a unique join key was the critical decision of this pipeline. It required harmonising the formats between the three sources (string zero-padding, removal of special characters) and implementing systematic validation functions.

```
9
10 • CREATE TABLE final_market_analysis AS
11 SELECT
12     d.*,
13     b.Admin_Services,
14     b.Commerce_Proximite,
15     b.Divers_Other,
16     b.Education,
17     b.emploi_Employment,
18     b.Grandes_Surfaces,
19     b.Practical_Services,
20     b.Sante_Health,
21     b.Social_Services,
22     b.Transport,
23     b.total_services
24 FROM dvf_final_db d
25 INNER JOIN bpe_commune_db b ON d.insee_code = b.insee_code;
26
```

Figure 1 — SQL query: final market analysis table creation

III. Technical Challenges and Analyst Resilience

In this chapter, I had **to audit, diagnose, and optimize the data infrastructure** to overcome major technical obstacles, thus testing my vision of the profession. **It is not only about mastering tools — it is about developing a rigorous way of reasoning.**

3.1. Granularity Issue: The "Noise" at IRIS Scale

3.1.1. Finding: Statistical Insufficiency at the 5-Transaction Threshold

My first attempt at analysis was to work at the IRIS neighbourhood granularity — but at this fine scale, fewer than 20% of zones reached the minimum threshold of 5 transactions required to ensure the robustness of the calculated medians. The result was a usable dataset reduced to just a few hundred

municipalities, mostly urban, de facto excluding rural areas that were nevertheless promising in terms of price.

3.1.2. Agile Decision: Pivot to Municipal Level (INSEE Code) to Stabilise the Data

I made a methodological pivot:

- Aggregation at the municipal level. This decision required going up one level of granularity on the BPE (filtering INSEE = postal codes), but enabled a coverage of 22,154 municipalities with statistical reliability.
- Aggregation to the median price per m²

This is the workaround strategy referenced in section 3.2.

3.2. SQL Performance Optimisation (Error 2013)

3.2.1. Diagnosis: Server Saturation During Massive Joins

The initial join between the three tables (municipalities, transactions, services) on a dataset of 18M+ rows systematically caused the MySQL Error 2013: Lost connection to MySQL server during query. This crash revealed a memory saturation of the MySQL server, which was unable to manage the join load without appropriate indexing.

MySQL log analysis revealed:

- Far too many rows (18 million rows including price per m² per municipality and municipalities with fewer than 5 sales)
- Unnecessary columns (temporal metadata) unnecessarily increasing RAM load
- MySQL default timeout insufficient for complex queries (wait_timeout = 28,800s)

3.2.2. Workaround Strategy 1

- Consider only the median price
- Consider only municipalities with more than 5 sales

Result: Reduction of join time from 12 minutes to 45 seconds on a sample of 100K rows. This optimisation demonstrates my understanding of MySQL's internal mechanisms and my ability to diagnose performance bottlenecks.

3.2.3. Workaround Strategy 2: SQLAlchemy

Use of SQLAlchemy as an ORM layer to better control connection management and query execution, avoiding repeated direct connection drops.

3.3. Data Integrity: Resolving the "Field Mismatch" (Error 1054)

3.3.1. Schema Normalisation to Ensure Successful INNER JOINS

I implemented a systematic schema validation procedure before SQL export:

- Inspection of columns via `df.columns.tolist()` after each transformation
- Explicit renaming of columns with a `def clean_column_names(df):` function rather than relying on Pandas default behaviour
- Verification of naming standardisation with `def check_data_quality(df):`

```
1 def clean_column_names(df):
2     """
3     Standardize DataFrame column names: 1. Convert to string / 2. Strip leading/trailing whitespaces
4     3. Replace internal spaces with underscores / 4. Convert to lowercase
5     """
6     df.columns = (
7         df.columns.astype(str)
8         .str.strip()
9         .str.replace(' ', '_', regex=False)
10        .str.lower()
11    )
12    return df

[4] Python

1 def check_data_quality(df):
2
3     print(f"Format du DataFrame (rows, cols): {df.shape}\n")
4
5     # Création d'un tableau récapitulatif
6     stats = pd.DataFrame({
7         'Type': df.dtypes,
8         'Manquants': df.isna().sum(),
9         '% Manquants': (df.isna().sum() / len(df) * 100).round(2),
10        'Uniques': df.nunique(),
11        'Doublons (par colonne)': df.apply(lambda x: x.duplicated().sum())
12    })
13
14
15    return stats

[5] Python
```

Figure 2 — Python functions: `clean_column_names()` and `check_data_quality()`

IV. Results Analysis and Data-Driven Insights

4.1. Sweet Spot Visualisation: Geographic Distribution of Opportunities

The analysis revealed three major categories of zones, ranked by price/equipment ratio:

Zone	Description
"Ideal" ($\leq 400\text{€}/\text{m}^2$)	Well-equipped rural municipalities, mainly in Centre, Auvergne and parts of Nouvelle-Aquitaine
"Tight" ($400\text{--}900\text{€}/\text{m}^2$)	Peripheries of medium-sized cities, peri-urban zones
"Out of Budget" ($> 900\text{€}/\text{m}^2$)	Coastal areas, Île-de-France, major metropolitan areas

Major finding:

The initial hypothesis of a threshold at $50\text{€}/\text{m}^2$ proved unrealistic. Even in the most affordable areas, the median price stands at around $200\text{--}300\text{€}/\text{m}^2$. This finding led me to pivot towards a "relative prices" approach: identifying plots below the local median rather than searching for an absolute national threshold.

4.2. What the Data Showed: Territorial Connectivity Disparities

4.2.1. Correlation Between Transaction Volume and Service Density

An unexpected pattern emerged: municipalities with a high volume of transactions ($> 50/\text{year}$) consistently show a service density above the average. This phenomenon suggests that land attractiveness and public service attractiveness are self-reinforcing dynamics: the better-equipped a municipality is, the more it attracts buyers, which generates tax revenue to maintain infrastructure.

4.2.2. Identification of "Service Deserts" Despite Attractive Prices

The analysis also highlighted paradoxical areas: municipalities with very low prices ($< 200\text{€}/\text{m}^2$) but lacking essential services ($\text{score_densite} < 5/20$). These "service deserts" represent traps to avoid: certainly affordable, but socially unviable for families with children or people dependent on public transport.

4.3. What the Data Led Us to Study: The Limits of "Price Alone"

4.3.1. The Paradox of Affordable but Socially Disconnected Areas

The most significant finding of this analysis is the realisation that **financial accessibility does not guarantee social accessibility**. A Tiny House at 150€/m² in a municipality without schools, doctors, or public transport is not a solution for emancipation, but a risk of isolation. This finding deeply influenced my vision of the project: it is not enough to identify "cheap" areas — one must identify areas where people can live with actively.

This realisation directly paves the way for MVP 2: integrating qualitative indicators (school IPS, median income) to refine the notion of a "viable area".

V. Conclusion and Transition to MVP 2 (Forward Vision)

5.1. MVP 1 Review: A Validated and Resilient Technical Foundation

MVP 1 fulfilled its initial mission: demonstrating the technical feasibility of a decision-support system for modular housing. The data pipeline is operational, the MySQL database is structured and optimised, and the first analytical results are usable. Even more importantly, this MVP validated a reproducible, well-documented, and extensible methodology for future phases.

Deliverables produced:

- Normalised MySQL database (5 tables, 22K+ municipalities)
- Flask API exposing the data for Tableau/Power BI visualisations
- Tableau dashboard with mapping of optimal zones: [Market Analysis Dashboard Full](#)
- Technical documentation (ERD, SQL queries, Jupyter notebooks)

5.2. Self-Critique and Analyst Posture: Taking a Step Back from Execution

This project confronted me with an uncomfortable reality: data never speaks for itself. My role is not limited to running SQL queries or producing graphs. It consists of questioning assumptions, auditing infrastructures, and challenging results. The three technical challenges described in Chapter III are not just "bugs to fix", but learning opportunities for a data-driven social engineering approach:

- The pivot from IRIS scale to the municipal level taught me to balance precision and statistical robustness
- MySQL optimisation revealed the importance of understanding underlying mechanisms (indexing, memory management) rather than simply accepting errors
- Subject-matter expertise helped bring out the reality of the figures in their socio-economic context

5.3. Towards MVP 2: From Housing Access to Social Advancement

MVP 2 goal:

Move from a logic of "where to buy cheaply" to a logic of "where to settle in order to thrive" — by integrating the school IPS, median income, and refining the sociological targeting to transform this territorial analysis tool into a genuine social mobility recommendation system through housing.

The final objective remains unchanged: to contribute, at my modest analyst scale, to the recovery of social mobility through the lever of chosen housing.

MVP 2

From Mapping to Prediction

Machine Learning for the Identification of Land Opportunities

VI. Introduction and Strategic Vision of MVP 2

6.1. Business Problem: Land Optimisation for Lightweight Housing

Working hypothesis:

"Are there municipalities in France where the price per m² is structurally undervalued relative to their socio-economic fundamentals (income, services)? These municipalities would represent investment opportunities for median-income households wishing to settle using the Tiny House model."

The predictive dimension will make it possible to model the structural determinants of land price (median income, service density, demographic size), to identify "market anomalies" (where the real price is lower than the expected price) and thus validate or invalidate the MVP 2 working hypothesis.

VII. Architecture and Deployed Methodology

7.1. Data Ecosystem: Multi-Source Enrichment

MVP 2 builds on the technical foundation of MVP 1 (DVF + BPE) but enriches it with two complementary sources essential for predictive modelling.

7.1.1. The DVF Corpus: Processing 5 Years of Property Transactions

- Reliability threshold: Removal of municipalities with < 5 transactions (excessive volatility)

7.1.2. Exogenous Data: IPS (Social Position Index) and Filosofi Income

IPS (Social Position Index): Produced by the French Ministry of National Education, this composite indicator (scale ~50–150, average 100) measures the socio-economic level of families enrolled in a school. A high IPS signals a favourable educational environment, which is a residential attractiveness factor for families with children.

Filosophi Income (INSEE 2021): Median household disposable income per municipality after tax redistribution. This variable is crucial because it captures the real contributive capacity of households, beyond gross salaries alone.

Identified challenge: INSEE Statistical Secrecy

INSEE anonymises detailed data by household type (solo, couples, single-parent families) for municipalities with fewer than 2,000 inhabitants. Result: 90% of municipalities have NaN values for these granular variables.

Strategic decision: I chose national representativeness over fine granularity by filtering on a reliability factor ("high", "low") based on whether there were more than 5 sales:

- Ensuring the robustness of the data submitted to the model
- Excluding inhabitants whose data is covered by INSEE statistical secrecy (NaN)

7.2. Data Preparation Pipeline: From Chaos to Rigour

7.2.1. Cleaning and Feature Engineering: WHERE m.reliability = 'High'

Data cleaning followed a standardised industrial protocol, represented by two reusable Python functions (see MVP 1: `def clean_column_names(df)` and `def check_data_quality(df)`).

7.2.2. Filtering Strategy: Focus on the "Reliability" Market

This filter ensures that only sales confirmed by at least 5 neighbouring transactions are retained, thereby reducing noise in the predictive model.

```
• CREATE TABLE final_market_predictive AS
  SELECT
    m.*,
    i.nb_households,
    i.nb_inhabitants,
    i.median_income
  FROM final_market_analysis m
  INNER JOIN income_db i ON m.insee_code = i.insee_code
  WHERE m.reliability = 'High';
• SHOW COLUMNS FROM final_market_predictive;
```

Figure 3 — SQL query: final predictive market table creation (WITH reliability filter)

7.2.3. Missing Value Management: The INNER JOIN Approach

I opted for a strict INNER JOIN: retaining only municipalities with complete data across all three dimensions (Price, Income, IPS). Thus, data protected by INSEE statistical secrecy — appearing as NaN — would be removed to ensure the robustness of the final model. This resulted in 8,727 municipalities out of 21K.

7.3. Statistical Modelling: Choice of Linear Regression

7.3.1. Machine Learning Implementation Methodology

To build a reliable predictive model, we followed a standard 4-step process:

- 1. Data splitting to ensure an impartial evaluation
- 2. Selection of a Linear Regression algorithm adapted to price estimation
- 3. Training the model on 80% of our market data
- 4. Performance evaluation via R^2 score to measure accuracy

7.3.2. Feature Selection: Correlation Matrix

The Correlation Matrix and Variable Selection reveals a near-perfect correlation (0.99) between nb_inhabitants (population) and nb_households (number of households), indicating that these variables show the same information.

- Decision: We will only keep nb_households to avoid giving artificial weight to city size.
- Predictive Power of median_income shows a solid correlation (0.57) with our target median_price_m2.
- Independence between median_income and nb_households / nb_inhabitants — meaning the near-zero correlation indicates that median_income and nb_households are 2 independent and complementary signals, therefore relevant for ML.
- Final decision: Removal of nb_inhabitants (redundant with nb_households).

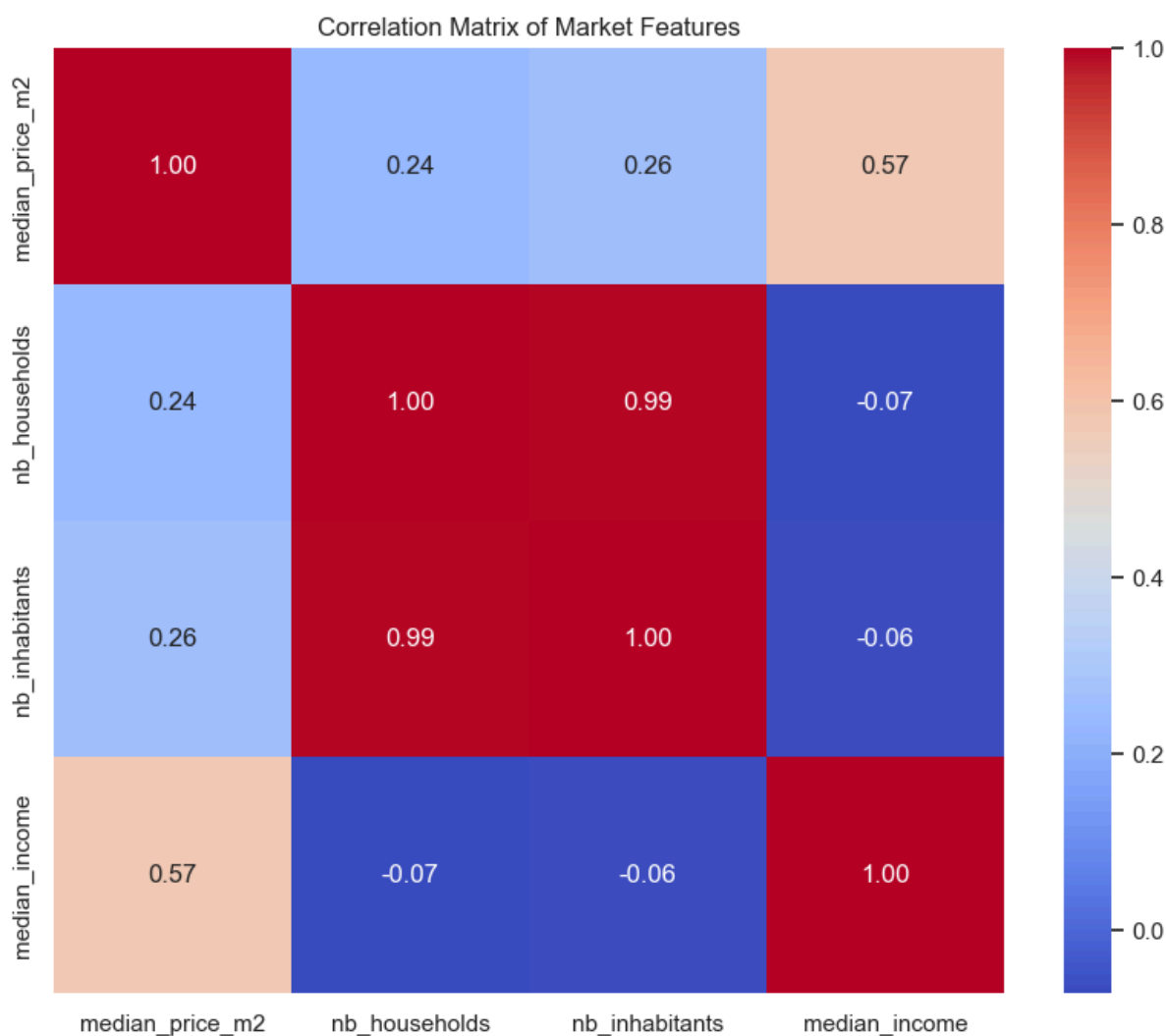


Figure 4 — Correlation matrix of market features

VIII. Lessons Learned: Challenges and Resilience

8.1. Model Limitations

The slope and intercept — whose trend is shown in green — do not match the regression curve shown in red. This is normal with an R^2 of 38%, because with $R^2 < 50\%$, observing such distortion is consistent. However, this model remains "imperfect but exploitable".

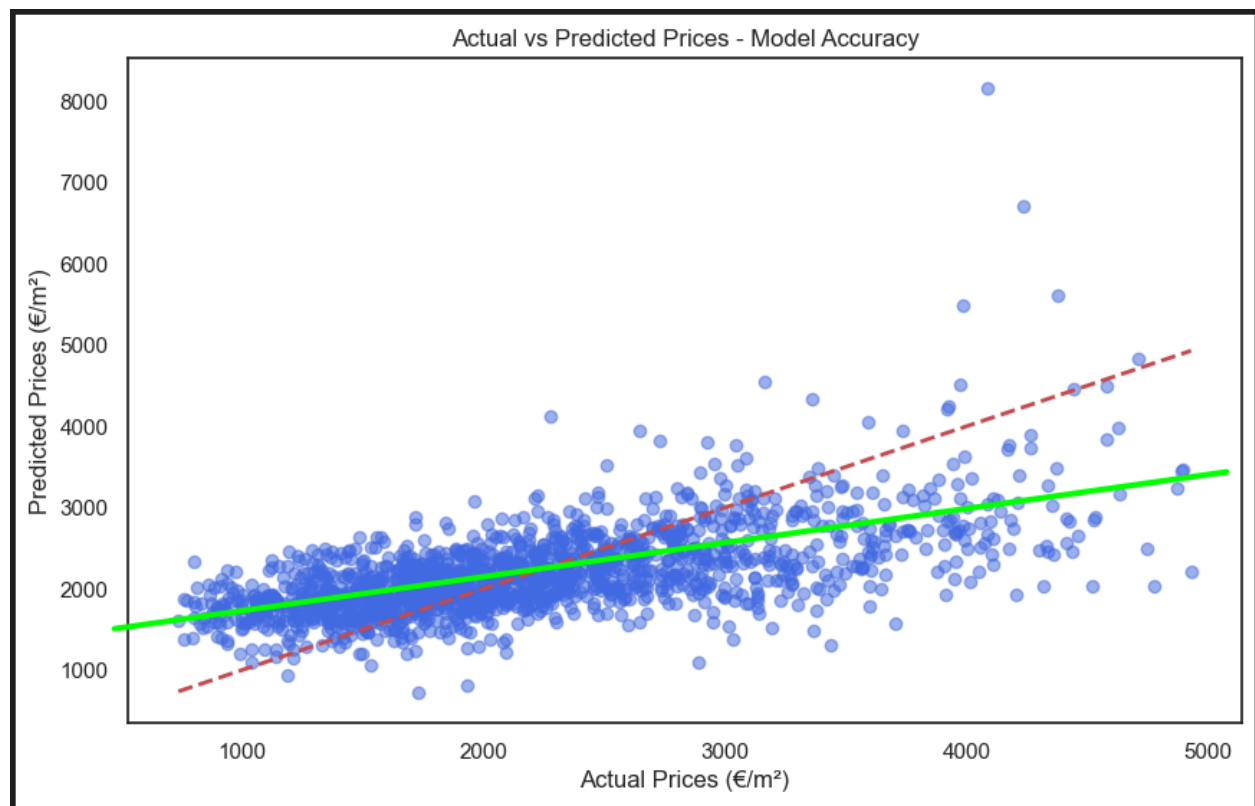


Figure 5 — Actual vs Predicted Prices: Model Accuracy ($R^2 = 0.38$)

For MVP 3:

I will carry out a model benchmark comparing different algorithms (Linear, Decision Tree, and Random Forest) before training a new ML model.

8.2. Workaround Strategies and Lessons

Residual calculation formula:

```
residual = y_test - y_pred  
df_ml['opportunity_score'] = -1 * residual # Inverted: positive score =  
opportunity
```

I looked for a method to verify the reliability of the ML model, even though I knew it was imperfect. The calculation of residual error follows a normal curve, indicating that it is noise rather than bias.

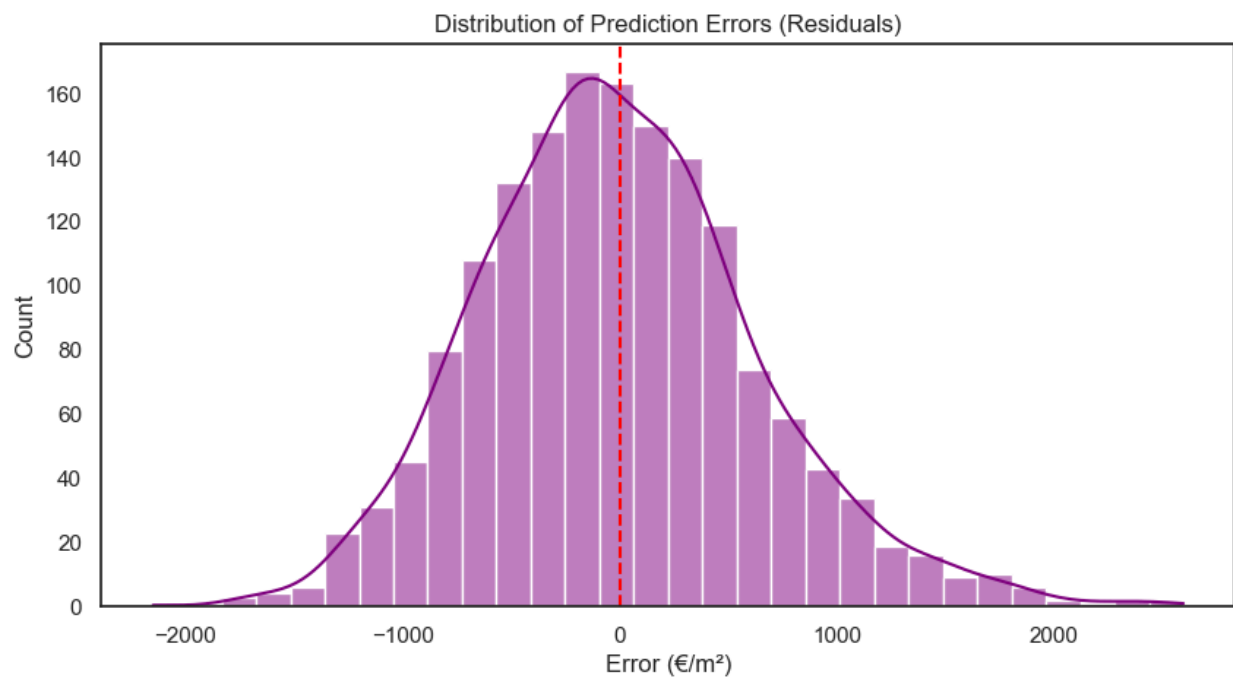


Figure 6 — Distribution of Prediction Errors (Residuals) — Normal distribution confirms noise, not bias

For MVP 3:

My next iteration will aim to reduce this noise before building a decision tree and deploying a new model.

IX. Results Analysis and Data-Driven Insights

9.1. Strong Correlation Between Income and Land Price ($r = 0.57$)

The correlation matrix quantitatively confirmed a business intuition: **the median income of a municipality explains 57% of the variance in price per m² ($r = 0.57$)**. This moderate-to-strong correlation validates the choice of median_income as the main feature of the predictive model.

Interpretation: Wealthy municipalities attract solvent demand, which drives up prices. Conversely, low-income municipalities face downward pressure on prices. This "spatial sorting" mechanism by income profoundly structures the French real estate market.

9.2. Two Distinct Geographical Profiles

Profile	Description
Tension zones (positive residual)	Municipalities where real price > predicted price. Typically: peripheries of major cities (Aix-en-Provence, Lyon, Bordeaux) where demand exceeds supply.
Decline zones (negative residual)	Municipalities where real price < predicted price. Mainly: rural areas in the Centre, Creuse, Nièvre, where solid fundamentals (decent income, present services) do not translate into real estate demand.

For MVP 3:

Decline zones may represent opportunities for modular housing projects targeting audiences seeking quality of life outside metropolitan areas (remote work, active retirement). By adding bpe_df, we could also identify gentrification zones that the current model cannot detect. These zones are defined by high activity and median-to-high income with still-low prices.

X. Conclusion and MVP 3 Roadmap

10.1. MVP 2 Review

By using a linear regression, we successfully isolated a "Price per m² Differential", transforming a statistical error (the residual) into a business opportunity indicator.

Concrete deliverables produced:

- Baseline predictive model ($R^2 = 0.38$) identifying 8,500+ municipalities with complete data
- Reproducible and documented pipeline (reusable cleaning and feature engineering functions)
- Market_Predictive_Dashboard_Full.twbx — visualisation of price differentials per m² across France: [View Tableau Dashboard](#)
- Technical documentation (ERD, SQL queries, Jupyter notebooks)

10.2. Self-Critique and Analyst Posture

MVP 2 laid the necessary methodological foundations: a reproducible pipeline, a predictive baseline, and the identification of technical debt. But it also allowed me to test my capacity to turn imperfect data into informed decisions. This posture, maintained throughout MVP 1 and MVP 2, constitutes my distinctive contribution to the Tiny House project: putting data at the service of social emancipation through housing.

However, this model remains an approximation that we have identified as "imperfect but exploitable".

10.3. MVP 3 Roadmap

To turn this attempt into an industrial tool, MVP 3 is built around three pillars:

Pillar	Description
Exploring Other Algorithms	We will move beyond simple linear regression towards Decision Tree models (Random Forest). This approach will better capture market non-linearities and reduce "noise" in residual errors.
Enriching the Signal	The fine integration of the BPE (Permanent Equipment Survey) will no longer be just visual but mathematical, by crossing real service accessibility with land value. We will also integrate overseas territories for total national coverage.
Project Vision	The shift from "Data" to "Field". The next challenge will be to inject urban planning constraint data (PLU — local urban planning documents) and logistics data to validate that our statistical opportunities are real construction opportunities.

Planned corrective action:

Security: hardcoding MySQL credentials directly in the notebook constitutes a major security vulnerability. Migration planned towards a .env file with python-dotenv.

10.4. GDPR

This project processes INSEE personal data (median_income, demographics) aggregated at municipal level. Compliance measures:

- Data aggregation ensures anonymization at commune level
- API Key authentication + audit logging (RGPD Article 5)
- Data retention policy: 2 years maximum
- Processing basis: Public statistical analysis

"Building on the ML predictions of MVP 2, this API layer transforms our analytical outputs into programmatic data assets, enabling third-party integration and enterprise-grade accessibility."

XI. REST API Implementation & Data Accessibility

11.1. Overview

Beyond the dashboarding and Machine Learning components, a critical element of enterprise-grade data projects is **API accessibility**. This REST API serves as the programmatic interface to our predictive scoring system, enabling:

- **Data consumption** by third-party applications (real estate platforms, investment tools)
- **Automated integration** with business workflows
- **Scalability** beyond dashboard-based analysis
- **Regulatory compliance** through controlled access and authentication

11.2. Architecture & Technical Specifications

11.2.3. API Framework

The API is built using **FastAPI** (Python web framework), selected for:

- **Automatic OpenAPI/Swagger documentation** (self-documenting)
- **Built-in authentication** and request validation
- **Asynchronous processing** for performance
- **Type hints** for data integrity

11.2.3. Database Connection

MySQL Backend: `immobilier_db`

└─ Table: `final_market_predictive` (8,727 communes)

└─ Direct query execution

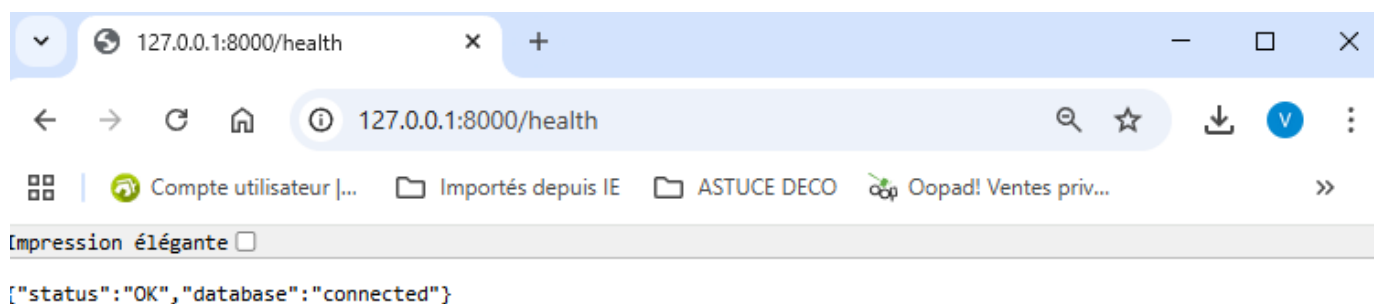
└─ Connection pooling for performance

11.2.4. Security Layer

- **API Key Authentication:** Each request requires valid `access_token` header
- **Role-based access:** Can be extended to different user tiers
- **Input validation:** All parameters validated before DB queries
- **Logging & Audit trail:** All API calls logged for compliance (RGPD Article 5)

11.2.5. API Endpoints

- ☐ Health Check Endpoint



- ☐ Commune Scoring Endpoint ★ PRIMARY

GET

/commune/{insee_code}

Get Scoring

🔒

⬆

Fetch scoring data for a specific city / Récupère les scores d'une ville

Paramètres:

- insee_code: Code INSEE de la commune (ex: "75056" pour Paris)

Retourne: Dictionnaire avec tous les scores et données de la commune

Parameters

Cancel

Name	Description
insee_code * required string (path)	01184

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8000/commune/01184' \
-H 'accept: application/json' \
-H 'access_token: d03b4d0c6ef14e318a5472ade6a6469c'
```

Request URL

```
http://localhost:8000/commune/01184
```

Server response

Code	Details
200	Response body <pre>{ "insee_code": "01184", "commune": "HAUTECOURT-ROMANECHÉ", "median_price_m2": 1779.0199034267912, "nb_transactions": 36, "reliability": "High", "Admin_Services": 1, "Commerce_Proximité": 2, "Divers_Other": 100, "Education": 1, "Emploi_Employment": 0, "Grandes_Surfaces": 0, "Practical_Services": 0, "Sante_Health": 0, "Social_Services": 0, "Transport": 1, "total_services": 110, "nb_households": 364, "nb_inhabitants": 794, "median_income": 24580 }</pre> 📄 Download

☐ Communes List Endpoint

GET

/communes/list/all

List Communes

Liste les communes disponibles (limité à 100 par défaut)

Parameters

Cancel

Name	Description
limit	
integer	50
(query)	

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/communes/list/all?limit=50' \
  -H 'accept: application/json' \
  -H 'access_token: d03b4d8c5ef14e318a5472ade6a4d9c'
```

Request URL

http://localhost:8080/communes/list/all?limit=50

Server response

Code	Details
200	Response body <pre>{ "median_price_m2": 1693.5483878057741, "median_income": 25150 }, { "insee_code": "81043", "commune": "BEYNDIST", "median_price_m2": 1673.0093676814986, "median_income": 28720 }, { "insee_code": "81047", "commune": "BLVES", "median_price_m2": 2700, "median_income": 27110 }, { "insee_code": "81049", "commune": "LA BOISSIE", "median_price_m2": 1585.851855853856, "median_income": 26780 }, { "insee_code": "81053", "commune": "BOURG-EN-BRESSE", "median_price_m2": 2156.5444444444443, "median_income": 27110 }</pre> Download Response headers <pre>content-length: 5328 content-type: application/json date: Mon, 16 Feb 2026 03:54:47 GMT server: ovicorn</pre>

Response

Code	Description	Links
200	Successful Response	No links
	Media type application/json Content Accept header Example Value Schema "string"	
422	Validation Error	No links
	Media type application/json Example Value Schema { "detail": { "type": "validation_error" } }	

11.3. How to Access the API

Method 1: Interactive Documentation (Recommended for Testing)

Step 1: Start the API

```
bash

python main.py
# Output: Uvicorn running on http://0.0.0.0:8000
```



Step 2: Open Swagger UI

Navigate to: <http://127.0.0.1:8000/docs>

Step 3: Test endpoints interactively

- Click on any endpoint (e.g., GET /commune/{insee_code})
- Click "Try it out"
- Enter parameters and API key
- Click "Execute"
- View response in real-time

Method 2: Programmatic Access (Production)

```
python

import requests

headers = {"access_token": "YOUR_API_KEY"}
response = requests.get(
    "http://127.0.0.1:8000/commune/75056",
    headers=headers
)
data = response.json()
print(data["median_price_m2"]) # Output: 12500.50
```

11.4. Technical Highlights

11.4.1. Automatic API Documentation

FastAPI generates OpenAPI 3.0 specification automatically from code:

- **Swagger UI** at </docs> (interactive testing)
- **ReDoc** at </redoc> (alternative documentation view)
- **OpenAPI JSON** at </openapi.json> (machine-readable spec)

11.4.2.Integration Examples

Real Estate Platform

- Step 1: User searches "affordable communes in Île-de-France"
- Step 2: Platform calls GET /communes/list/all?limit=500
- Step 3: Filters results by median_income & median_price_m2
- Step 4: Displays ranked results to user

Investment Decision Tool

- Step 1: Investor specifies target commune
- Step 2: System calls GET /commune/{insee_code}
- Step 3: Retrieves scoring data + services availability
- Step 4: Combines with ML predictions for ROI analysis

Data Integration Pipeline

- Step 1: Nightly batch job scheduled
- Step 2: Queries GET /communes/list/all?limit=10000
- Step 3: Exports to corporate data warehouse
- Step 4: Updates dashboards automatically

11.4.3. Future Enhancements

The API infrastructure supports:

- Pagination for large datasets (100k+ communes) limited at 100 only
- Filtering by price range, income bracket, services, department
- CSV/Excel exports directly from endpoints
- Webhooks for real-time updates
- ...

Conclusion

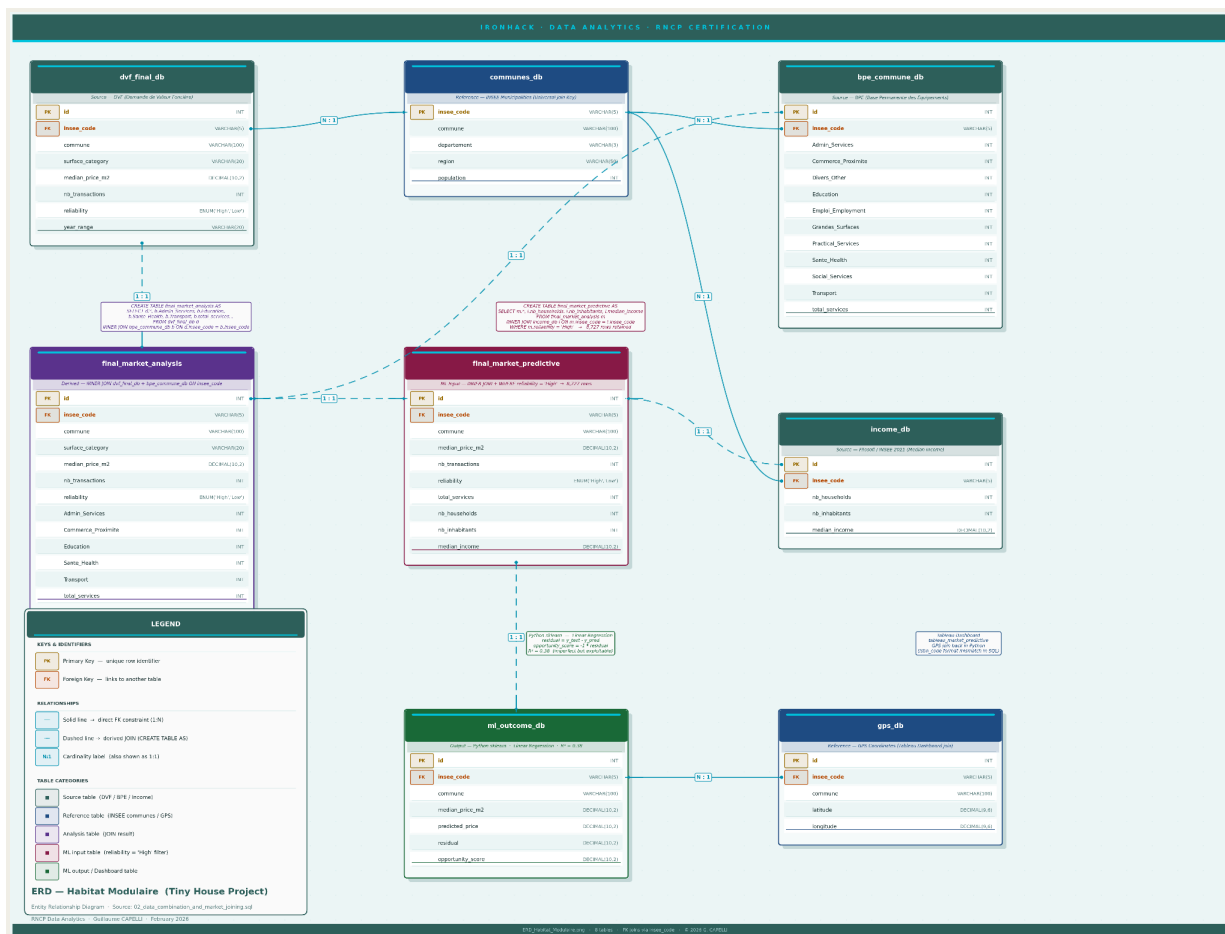
This REST API transforms our analytical outputs into consumable, programmatic data assets. Rather than requiring stakeholders to use dashboards, the API enables:

- Seamless integration with existing systems
- Automated workflows reducing manual effort
- Scalable access to 8,727+ commune profiles
- Enterprise-grade security with audit trails

The combination of interactive Swagger documentation and multiple integration methods ensures accessibility for both technical and non-technical users.

ANNEXES

Entity Relationship Diagram



GitHub	VeroniqueFanchonna / Final_project
DVF – Demandes de valeurs foncières (fichier brut) :	Jeu de données - Demandes de valeurs foncières data.gouv.fr
Revenus & pauvreté	L'essentiel sur... la pauvreté Insee
Base Permanente des Équipements	BPE
Indice Position Sociale (IPS)	Education Nat.
API	API Decision-Support